

International Cybersecurity and Digital Forensics Academy (ICDFA)

CIP-B101: Basic Computer Skills for Digital Forensics

Lab 1B: PC System Fundamentals, Disk Sectors and Partition Analysis

Student Name: Almustapha Yusuf

Student ID: fsdw11343@students.icdfa.edu.ng

Date: August 9, 2026

Platform: Kali Linux (VMware Virtual Machine)

Observing, recording, calculating, verifying, and documenting system findings in a repeatable way.

Introduction

This lab introduces the basic computer system knowledge required for beginner digital forensic work. The objective is to document a computer system safely, identify possible evidence layers, inspect disk and partition details, calculate partition sizes from sector information, explain the boot process, and identify file systems without altering the computer. All commands used in this lab are read-only observation commands. No partitions were created, deleted, or modified. The examination was performed on a Kali Linux virtual machine running within VMware, providing a safe training environment that mirrors real-world forensic system profiling tasks.

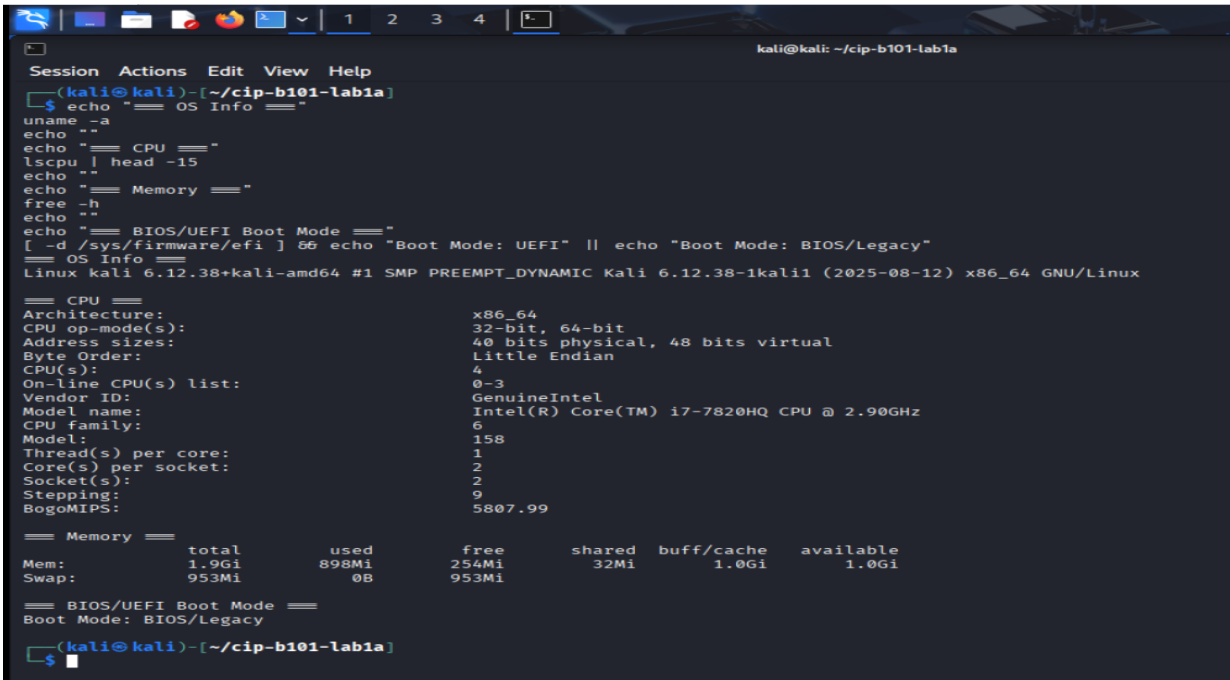
Tools Used

Operating System: Kali Linux (kernel 6.12.38+kali-amd64) running on VMware VM. Commands: uname, lscpu, free, lsblk, fdisk, parted, blkid, df, ps. All commands were executed in a standard user terminal with sudo access for disk inspection commands. Python 3 was used for disk sector size calculations.

Part A: System Identification and Evidence Layer Mapping

A.1 System Specification

Component	Value
OS	Kali Linux (6.12.38+kali-amd64, x86_64)
Kernel	6.12.38-1kali1 (2025-08-12)
Architecture	x86_64 (64-bit), Little Endian
CPU	Intel Core i7-7820HQ @ 2.90GHz
CPU Cores	4 (2 sockets, 2 cores each, 1 thread per core)
Total RAM	1.9 GiB
Swap	953 MiB
Boot Mode	BIOS/Legacy (no /sys/firmware/efi)
Disk	VMware Virtual, 80.09 GiB
File System	ext4 (block size 4096)
Partition Table	MBR (dos) - single primary partition



```
kali@kali: ~/cip-b101-lab1a
Session Actions Edit View Help
(kali@kali)-[~/cip-b101-lab1a]
$ echo "=== OS Info ==="
uname --a
echo --
echo "=== CPU ==="
lscpu | head -15
echo --
echo "=== Memory ==="
free -h
echo --
echo "=== BIOS/UEFI Boot Mode ==="
[ -d /sys/firmware/efi ] && echo "Boot Mode: UEFI" || echo "Boot Mode: BIOS/Legacy"
=== OS Info ===
Linux kali 6.12.38+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12) x86_64 GNU/Linux

=== CPU ===
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               40 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      4
On-line CPU(s) list:         0-3
Vendor ID:                   GenuineIntel
Model name:                  Intel(R) Core(TM) i7-7820HQ CPU @ 2.90GHz
CPU family:                  6
Model:                      158
Thread(s) per core:          1
Core(s) per socket:          2
Socket(s):                   2
Stepping:                    9
BogoMIPS:                    5807.99

=== Memory ===
              total        used         free      shared  buff/cache   available
Mem:           1.9Gi         898Mi        254Mi         32Mi         1.0Gi         1.0Gi
Swap:           953Mi           0B          953Mi

=== BIOS/UEFI Boot Mode ===
Boot Mode: BIOS/Legacy

(kali@kali)-[~/cip-b101-lab1a]
$
```

Figure 1: System information - OS, CPU, memory, and boot mode

```
vmware-a... x
kali@kali: ~/cip-b101-lab1a

Session Actions Edit View Help
(kali@kali)-[~/cip-b101-lab1a]
$ echo "=== Disk Drives ==="
lsblk -o NAME,SIZE,TYPE,MODEL
echo ""
echo "=== File Systems ==="
lsblk -f
echo ""
echo "=== Disk Usage ==="
df -hT
=== Disk Drives ===
NAME        SIZE TYPE MODEL
sda         80.1G disk VMware Virtual S
└─sda1 80.1G part

=== File Systems ===
NAME      FSTYPE FSVER LABEL UUID                               FSAVAIL FSUSE% MOUNTPOINTS
sda
└─sda1 ext4   1.0   root  4111f04f-67ad-4d5d-a01b-b6cc151feb35  48.3G   33% /

=== Disk Usage ===
Filesystem      Type      Size  Used Avail Use% Mounted on
udev            devtmpfs  916M   0    916M   0% /dev
tmpfs           tmpfs     197M   1.3M 196M   1% /run
/dev/sda1       ext4      79G   26G   49G   35% /
tmpfs           tmpfs     983M   4.0K 983M   1% /dev/shm
tmpfs           tmpfs     5.0M   0    5.0M   0% /run/lock
tmpfs           tmpfs     1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
tmpfs           tmpfs     983M  204K 983M   1% /tmp
tmpfs           tmpfs     1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           tmpfs     197M  124K 197M   1% /run/user/1000

(kali@kali)-[~/cip-b101-lab1a]
$
```

Figure 2: Storage information - disk drives, file systems, and disk usage

A.2 Evidence Layer Mapping

Digital evidence can exist at multiple system layers simultaneously. A single user action generates artifacts across several layers, which is what makes digital forensics powerful. Even if evidence is deleted from one layer, remnants may persist in others. The following table maps the five key system layers to the types of evidence each can produce, where such evidence is found, and why it matters in a forensic investigation.

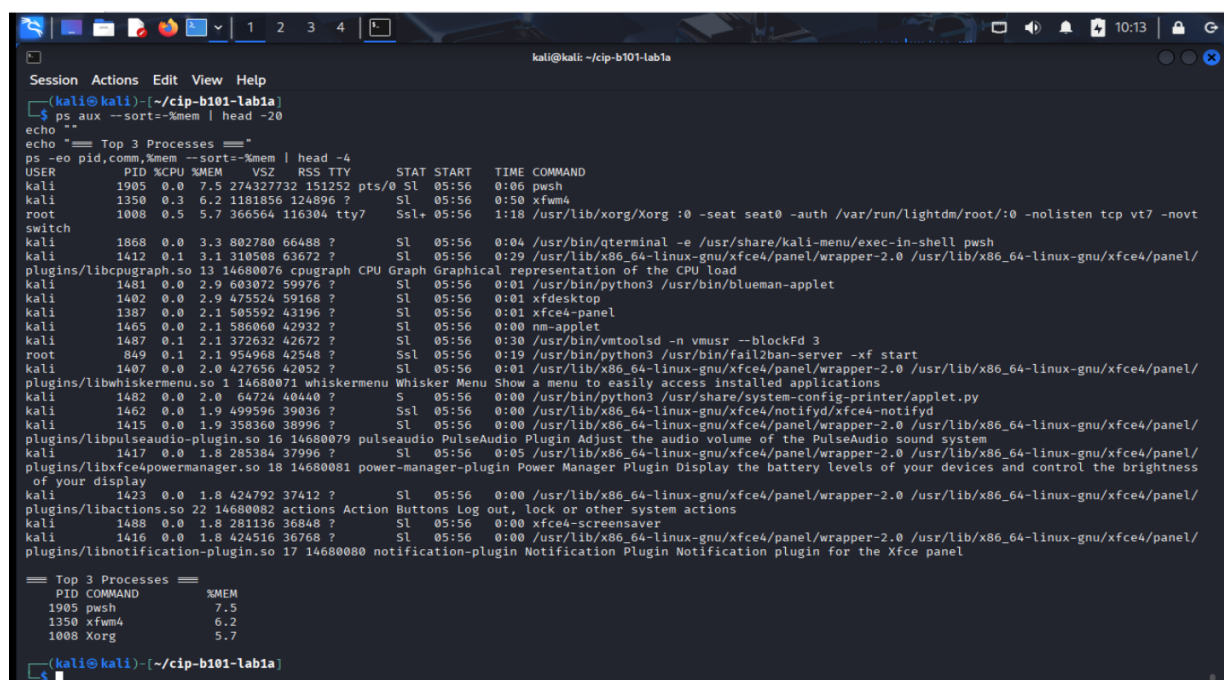
Layer	Example Evidence	Where Found	Why It Matters
User	Opened files, search terms, commands	.bash_history, browser history	Links user to system activity
Application	Document metadata, cache, logs	App databases, config files	Shows software-created evidence
OS	Event logs, installed programs	/var/log, journal, /etc/passwd	Shows system events and accounts
Disk	Deleted files, unallocated space	Slack space, unallocated clusters	Supports recovery and timelines
Memory/Process	Running malware, PIDs, connections	RAM capture, /proc, ps	Shows live system behavior

Part B: Program, Process and Thread Investigation

A program is a static file stored on disk that contains executable instructions. A process is a running instance of a program that has been loaded into memory and assigned a PID, memory space, and system resources by the operating system. A thread is the smallest unit of execution within a process; a single process may contain multiple threads that share the same memory space but execute independently. In the context of digital forensics, process information is critical during malware analysis and live forensic triage because it reveals what software is currently active on the system, which processes may be suspicious or unauthorized, and what network connections are associated with each process. An analyst examining a compromised system would immediately check running processes to identify malicious executables, unusual parent-child process relationships, or processes consuming abnormal resources.

B.1 Observed Processes (Selected Top 3)

PID	Process Name	Memory %	Likely Purpose
1905	pwsh	7.5%	PowerShell 7 terminal - user interactive session
1008	Xorg	5.7%	X11 display server providing graphical environment
1350	xfwm4	6.2%	Xfce window manager managing all graphical windows



```
Session Actions Edit View Help
kali@kali)~[~/cip-b101-lab1a]
$ ps aux --sort=-%mem | head -20
echo ""
echo "=== Top 3 Processes ==="
ps -eo pid,comm,%mem --sort=-%mem | head -4
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
kali      1905  0.0  7.5 27432732 151252 pts/0    SL   05:56   0:06 pwsh
kali      1350  0.3  6.2 1181856 124896 ?        SL   05:56   0:50 xfwm4
root      1008  0.5  5.7 366564 116304 tty7      Ssl+ 05:56   1:18 /usr/lib/xorg/Xorg :0 -seat seat0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -novt
switch
kali      1868  0.0  3.3 802780 66488 ?        SL   05:56   0:04 /usr/bin/qterminal -e /usr/share/kali-menu/exec-in-shell pwsh
kali      1412  0.1  3.1 310508 63672 ?        SL   05:56   0:29 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/
plugins/libcpugraph.so 13 14680076 cpugraph CPU Graph Graphical representation of the CPU load
kali      1481  0.0  2.9 603072 59976 ?        SL   05:56   0:01 /usr/bin/python3 /usr/bin/blueman-applet
kali      1402  0.0  2.9 475524 59168 ?        SL   05:56   0:01 xfdesktop
kali      1387  0.0  2.1 505592 43196 ?        SL   05:56   0:01 xfce4-panel
kali      1465  0.0  2.1 586060 42932 ?        SL   05:56   0:00 nm-applet
kali      1487  0.1  2.1 372632 42672 ?        SL   05:56   0:30 /usr/bin/vmtoolsd -n vmusr --blockFd 3
root      849  0.1  2.1 954968 42548 ?        Ssl  05:56   0:19 /usr/bin/python3 /usr/bin/fail2ban-server -xf start
kali      1407  0.0  2.0 427656 42052 ?        SL   05:56   0:01 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/
plugins/libwhiskermenu.so 1 14680071 whiskermenu Whisker Menu Show a menu to easily access installed applications
kali      1482  0.0  2.0 64724 40440 ?        S   05:56   0:00 /usr/bin/python3 /usr/share/system-config-printer/applet.py
kali      1462  0.0  1.9 499596 39036 ?        Ssl  05:56   0:00 /usr/lib/x86_64-linux-gnu/xfce4/notifyd/xfce4-notifyd
kali      1415  0.0  1.9 358360 38996 ?        SL   05:56   0:00 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/
plugins/libpulseaudio-plugin.so 16 14680079 pulseaudio PulseAudio Plugin Adjust the audio volume of the PulseAudio sound system
kali      1417  0.0  1.8 285384 37996 ?        SL   05:56   0:05 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/
plugins/libxfce4powermanager.so 18 14680081 power-manager-plugin Power Manager Plugin Display the battery levels of your devices and control the brightness
of your display
kali      1423  0.0  1.8 424792 37412 ?        SL   05:56   0:00 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/
plugins/libactions.so 22 14680082 actions Action Buttons Log out, lock or other system actions
kali      1488  0.0  1.8 281136 36848 ?        SL   05:56   0:00 xfce4-screensaver
kali      1416  0.0  1.8 424516 36768 ?        SL   05:56   0:00 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/
plugins/libnotification-plugin.so 17 14680080 notification-plugin Notification Plugin Notification plugin for the Xfce panel

=== Top 3 Processes ===
PID COMMAND      %MEM
1905 pwsh            7.5
1350 xfwm4           6.2
1008 Xorg            5.7

kali@kali)~[~/cip-b101-lab1a]
$
```

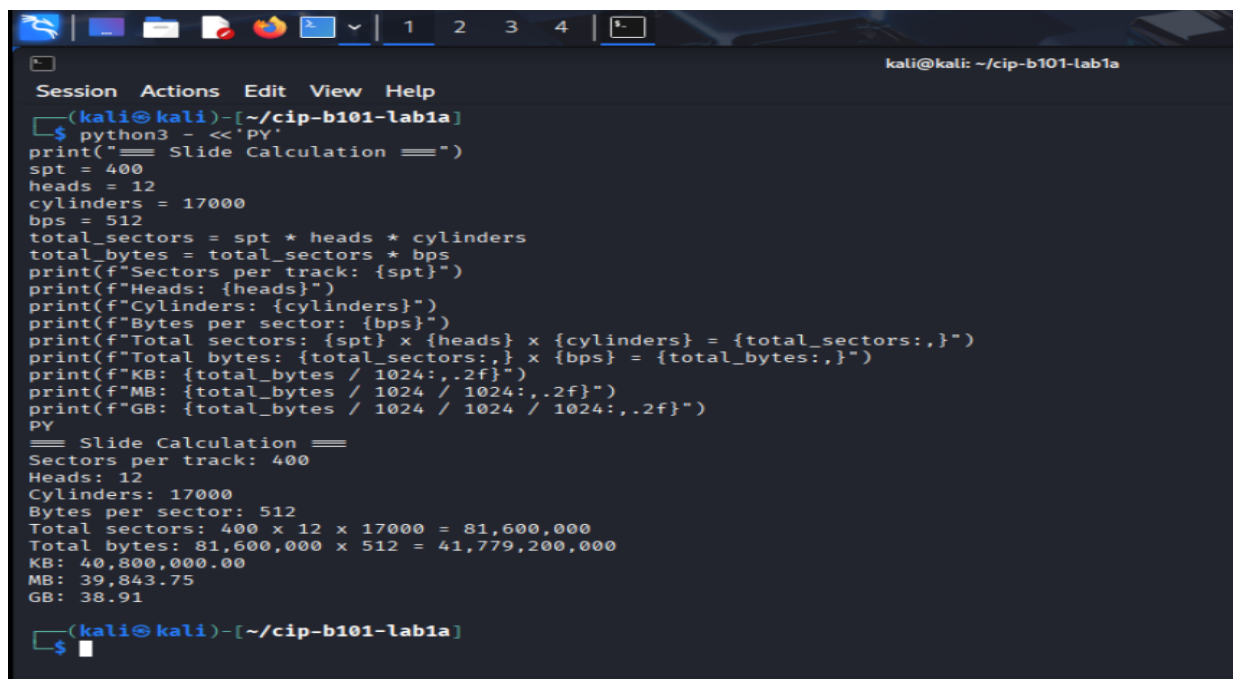
Figure 3: Running processes sorted by memory usage

Part C: Disk Sector and Size Calculation

C.1 Slide Calculation

The course slide provides values for a traditional CHS (Cylinder-Head-Sector) disk geometry. The total disk size is calculated by multiplying sectors per track by the number of heads by the number of cylinders, then multiplying by 512 bytes per sector. This formula demonstrates how physical disk geometry parameters translate into the total storage capacity.

Item	Formula / Value	Answer
Total sectors	$400 \times 12 \times 17,000$	81,600,000 sectors
Total bytes	$81,600,000 \times 512$	41,779,200,000 bytes
Kilobytes	$41,779,200,000 / 1,024$	40,800,000.00 KB
Megabytes	$40,800,000 / 1,024$	39,843.75 MB
Gigabytes	$39,843.75 / 1,024$	38.91 GB



```
(kali@kali)~[~/cip-b101-lab1a]
$ python3 - <<'PY'
print("== Slide Calculation ==")
spt = 400
heads = 12
cylinders = 17000
bps = 512
total_sectors = spt * heads * cylinders
total_bytes = total_sectors * bps
print(f"Sectors per track: {spt}")
print(f"Heads: {heads}")
print(f"Cylinders: {cylinders}")
print(f"Bytes per sector: {bps}")
print(f"Total sectors: {spt} x {heads} x {cylinders} = {total_sectors:,}")
print(f"Total bytes: {total_sectors:,} x {bps} = {total_bytes:,}")
print(f"KB: {total_bytes / 1024:,.2f}")
print(f"MB: {total_bytes / 1024 / 1024:,.2f}")
print(f"GB: {total_bytes / 1024 / 1024 / 1024:,.2f}")
PY
== Slide Calculation ==
Sectors per track: 400
Heads: 12
Cylinders: 17000
Bytes per sector: 512
Total sectors: 400 x 12 x 17000 = 81,600,000
Total bytes: 81,600,000 x 512 = 41,779,200,000
KB: 40,800,000.00
MB: 39,843.75
GB: 38.91
(kali@kali)~[~/cip-b101-lab1a]
$
```

Figure 4: Slide calculation - Python validation output

C.2 System Partition Size Calculation

Using the fdisk partition table for /dev/sda, the following calculation was performed for partition 1. Note: This system has only one partition (sda1). No partition 2 exists on this VM. The MBR gap occupies sectors 0-2047 (1,048,576 bytes = 1 MiB), which is standard modern alignment practice.

Parameter	Value
Device	/dev/sda1
Start Sector	2,048
End Sector	167,968,749
Sector Count	167,966,702
Sector Size	512 bytes
Size in Bytes	$167,966,702 \times 512 = 85,998,951,424$

Parameter	Value
Size in GiB	$85,998,951,424 / 1,024 / 1,024 / 1,024 = 80.09 \text{ GiB}$
File System	ext4
OS Reported Size	79G (df -hT)
Difference	~1 GiB due to ext4 overhead (reserved blocks, journal, inodes, metadata)

Partition 2 Note: This virtual machine has only a single partition (sda1). In a typical forensic examination of a multi-partition system, the same calculation methodology would be applied to partition 2 using its start sector, end sector, and sector count from the partition table. The formula remains: $\text{partition_size_GiB} = \text{sector_count} \times 512 / 1,024 / 1,024 / 1,024$.

```

kali@kali: ~/cip-b101-lab1a
Session Actions Edit View Help
(kali@kali)-[~/cip-b101-lab1a]
$ sudo fdisk -l /dev/sda 2>/dev/null || sudo fdisk -l /dev/nvme0n1 2>/dev/null
Disk /dev/sda: 80.09 GiB, 86000000000 bytes, 167968750 sectors
Disk model: VMware Virtual S
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xfebdb644

Device      Boot Start      End  Sectors  Size Id Type
/dev/sda1   *    2048 167968749 167966702  80.1G 83 Linux

(kali@kali)-[~/cip-b101-lab1a]
$

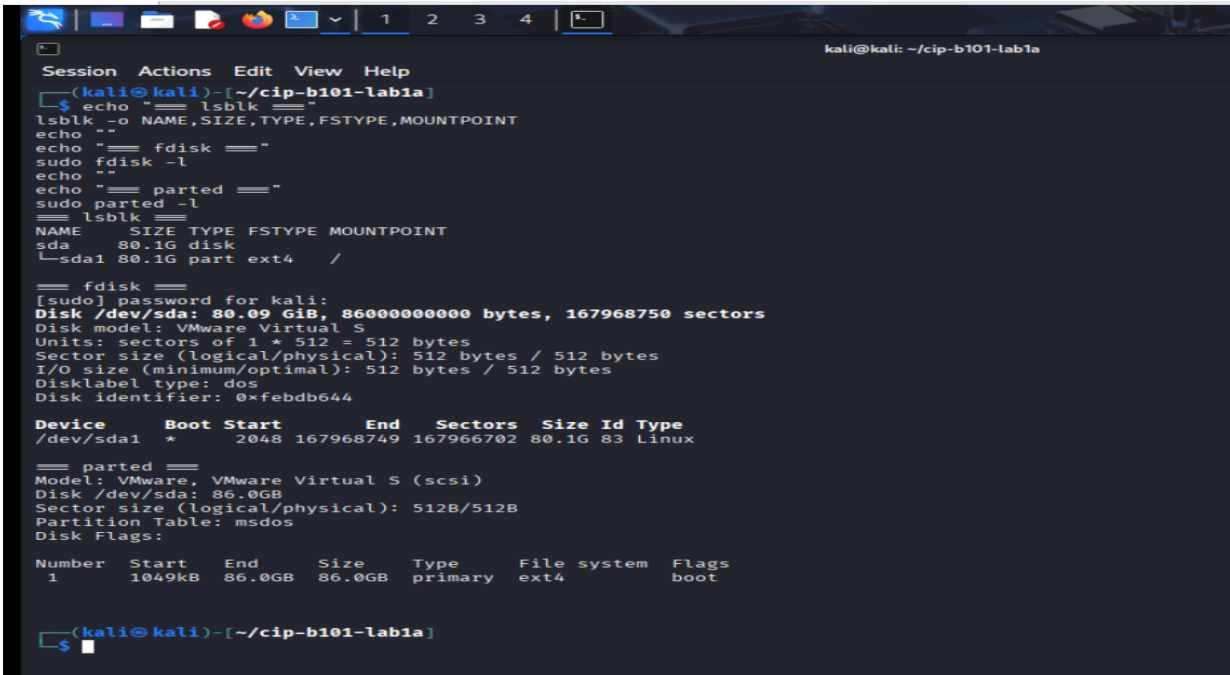
```

Figure 5: fdisk output showing partition table with sector details

Part D: Safe Partition Table Inspection

Partition table inspection was performed using three complementary read-only tools: lsblk for a quick overview of block devices and their relationships, fdisk -l for detailed sector-level partition information, and parted -l for an alternative view including human-readable sizes. All three tools confirmed a single 80.09 GiB primary partition using MBR (dos) partitioning with an ext4 file system. The disklabel type is dos, indicating the legacy MBR scheme. The partition is flagged as bootable. No destructive commands (clean, format, delete, mkfs) were executed.

Tool	Key Finding
lsblk	sda 80.1G disk with sda1 80.1G ext4 partition mounted at /
fdisk -l	167,968,750 total sectors, MBR/dos label, partition ID 83 (Linux)
parted -l	86.0GB disk, msdos table, 1049kB to 86.0GB primary ext4 boot



```
Session Actions Edit View Help
kali@kali: ~/cip-b101-lab1a
(kali@kali)-[~/cip-b101-lab1a]
$ echo "=== lsblk ==="
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT
echo ""
echo "=== fdisk ==="
sudo fdisk -l
echo ""
echo "=== parted ==="
sudo parted -l
echo "=== lsblk ==="
NAME        SIZE TYPE FSTYPE MOUNTPOINT
sda          80.1G disk
└─sda1 80.1G part ext4 /

=== fdisk ===
[sudo] password for kali:
Disk /dev/sda: 80.09 GiB, 86000000000 bytes, 167968750 sectors
Disk model: VMware Virtual S
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xfebdb644

Device      Boot Start      End  Sectors  Size Id Type
/dev/sda1   *      2048 167968749 167966702  80.1G 83 Linux

=== parted ===
Model: VMware, VMware Virtual S (scsi)
Disk /dev/sda: 86.0GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type    File system  Flags
  1      1049kB  86.0GB  86.0GB  primary ext4          boot

(kali@kali)-[~/cip-b101-lab1a]
$
```

Figure 6: Partition table inspection - lsblk, fdisk, and parted output

Part E: MBR, GPT and Boot Process

E.1 Boot Process (This System: BIOS/Legacy Mode)

Stage	What Happens	Forensic Relevance
1. Power-On / POST	Hardware initialized and checked	Failure/tampering affects evidence access
2. BIOS Firmware	Boot device order applied from CMOS	Boot mode explains partition style
3. MBR (Sector 0)	Boot code loads from first 512 bytes	MBR damage affects partition interpretation
4. Boot Loader (GRUB)	Kernel loaded from boot partition	May reveal multi-boot or hidden OS
5. Kernel Loading	vmlinux and initrd loaded into memory	Kernel parameters and init state captured
6. OS Control	systemd starts, services launch, login	Logs and user activity begin here

This system uses BIOS/Legacy boot mode, confirmed by the absence of the `/sys/firmware/efi` directory. The partition table is MBR (dos) with a single ext4 partition flagged as bootable. There is no EFI System Partition (ESP), which is consistent with legacy boot.

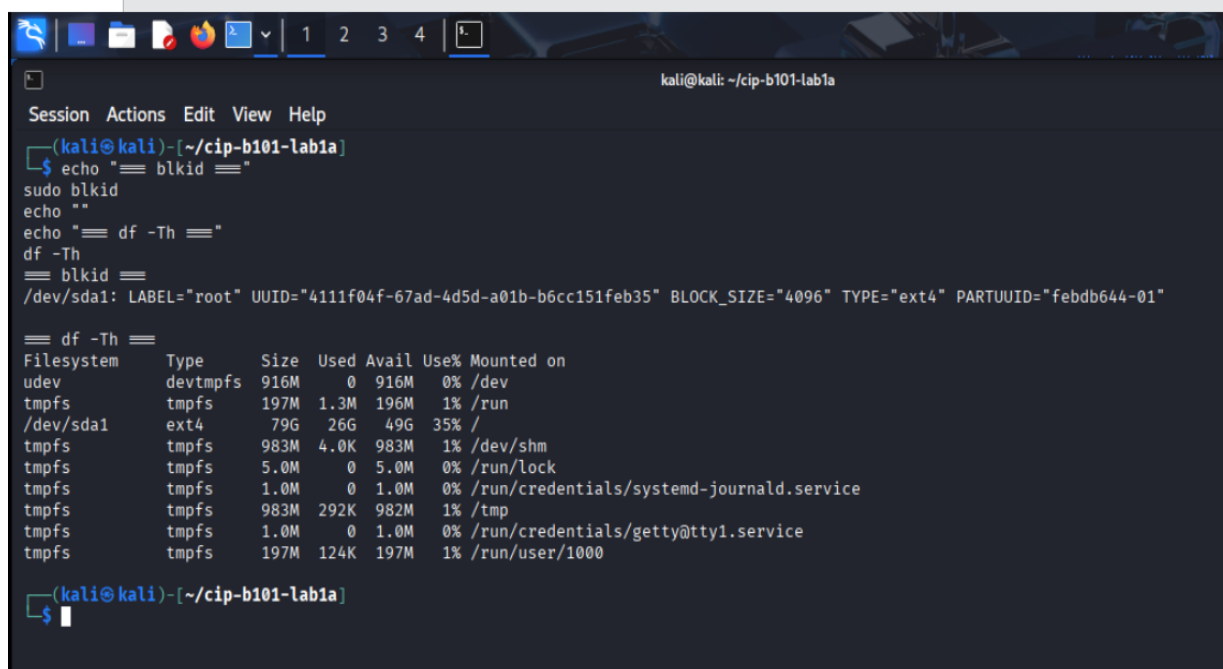
E.2 MBR vs GPT Comparison

Feature	MBR	GPT
Age/Design	Older, 1980s	Modern, UEFI specification
Max Disk Size	~2 TB	~9.4 ZB
Max Partitions	4 primary (or 3 + extended)	128 (default)
Partition Table Location	Sector 0 (first 64 bytes)	LBA 1 + backup at disk end
Integrity	No checksums or backup	CRC32 checksums, backup header
Forensic Issue	Sector 0 damage is critical	Backup aids recovery
Boot Mode	BIOS/Legacy	UEFI (can support legacy)
This System	Yes - dos disklabel	No

Part F: File System Identification

The primary file system on this system is ext4, identified through multiple sources: lsblk -f shows FSTYPE ext4, blkid confirms TYPE=ext4 with BLOCK_SIZE=4096, and df -Th shows Type ext4. The ext4 (Fourth Extended Filesystem) is the default file system for most Linux distributions. It uses inode-based metadata, supports journaling for crash recovery, and maintains timestamps for file access, modification, and inode change (atime, mtime, ctime). These timestamps are essential for building forensic timelines.

File System	Common OS	Forensic Importance
FAT/FAT32	USB drives, older systems	Simple structure, limited metadata, no journaling
NTFS	Modern Windows	MFT entries, timestamps, permissions, USN Journal
ext4	Linux systems	Inode metadata, journal, timeline analysis
XFS/JFS/btrfs	Linux/Unix	Advanced features affect recovery and metadata
HFS+	Older macOS	Catalog file contains file records

A screenshot of a Kali Linux terminal window. The window title is 'kali@kali: ~/cip-b101-lab1a'. The terminal shows a series of commands and their outputs. First, the user runs 'echo "=== blkid ==="', followed by 'sudo blkid', which outputs the blkid information for /dev/sda1. Then, the user runs 'echo "=== df -Th ==="', followed by 'df -Th', which outputs a table of disk usage for all mounted file systems. The terminal output is as follows:

```
kali@kali: ~/cip-b101-lab1a
Session Actions Edit View Help
(kali@kali)-[~/cip-b101-lab1a]
└─$ echo "=== blkid ==="
sudo blkid
echo ""
echo "=== df -Th ==="
df -Th
=== blkid ===
/dev/sda1: LABEL="root" UUID="4111f04f-67ad-4d5d-a01b-b6cc151feb35" BLOCK_SIZE="4096" TYPE="ext4" PARTUUID="febdb644-01"

=== df -Th ===
Filesystem      Type      Size  Used Avail Use% Mounted on
udev            devtmpfs  916M   0    916M   0% /dev
tmpfs           tmpfs     197M  1.3M  196M   1% /run
/dev/sda1       ext4      79G   26G   49G   35% /
tmpfs           tmpfs     983M  4.0K  983M   1% /dev/shm
tmpfs           tmpfs     5.0M   0    5.0M   0% /run/lock
tmpfs           tmpfs     1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
tmpfs           tmpfs     983M  292K  982M   1% /tmp
tmpfs           tmpfs     1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           tmpfs     197M  124K  197M   1% /run/user/1000

(kali@kali)-[~/cip-b101-lab1a]
└─$
```

Figure 7: File system identification - blkid and df -Th output

File systems are critical in digital forensics because they maintain the metadata structure that records when files were created, modified, and accessed. Understanding the file system type is required to correctly parse metadata, recover deleted files, and interpret journal entries. For example, ext4 uses inodes that may persist after file deletion until overwritten, enabling recovery. The journal records recent file system operations, which can reveal evidence of file deletion or renaming even after the original metadata has been modified.

Conclusion

This lab provided hands-on experience in system profiling, a foundational skill in digital forensics. By using read-only commands to document system specifications, inspect partition tables, and calculate disk sizes from sector information, I gained practical understanding of how digital evidence is stored and structured at the hardware and file system levels. The evidence layer mapping exercise demonstrated that a single user action generates artifacts across multiple system layers, reinforcing the principle that thorough forensic analysis must examine all layers, not just one. The disk sector calculations highlighted the importance of understanding the relationship between physical disk geometry (or its modern equivalent in logical block addressing) and the storage capacities reported by operating systems. The boot process documentation and MBR/GPT comparison showed how the choice of firmware and partition scheme affects the types of forensic artifacts available. Finally, file system identification reinforced that knowing the file system type is a prerequisite for any forensic tool selection or data interpretation. All work was performed using safe, non-destructive commands in compliance with the lab safety requirements.

Screenshot Checklist

No.	Screenshot Evidence	Included?
1	System information (OS, CPU, memory, boot mode)	Yes (Figure 1)
2	Disk drive model and size output	Yes (Figure 2)
3	Logical volume/file system output	Yes (Figure 2)
4	Running process/PID output	Yes (Figure 3)
5	Partition table (lsblk, fdisk, parted)	Yes (Figure 6)
6	Partition 1 calculation evidence	Yes (Figure 5)
7	Partition 2 - N/A (single partition VM)	Explained in report
8	Boot mode evidence (BIOS/UEFI)	Yes (Figure 1)
9	File system identification evidence	Yes (Figure 7)
10	Final report summary/conclusion	Yes (this section)